SEGUROS BOLÍVAR PROCESO TI

Prueba Técnica - Desarrollador TI / Sénior

Informe técnico consolidado: arquitectura, implementación, datos, versionamiento y liderazgo

Presentación del entregable

• Documento único que integra los cinco módulos requeridos; una matriz de cumplimiento enlaza cada requerimiento del enunciado con su respuesta.

• Módulo 2 implementado en Java 21 y Spring Boot 3.4.3, versionado en GitHub; 18 pruebas automatizadas ejecutadas en verde.

• Cada decisión incluye su justificación, sus compromisos (trade-offs) y los supuestos adoptados donde el enunciado no define el detalle.

Candidato:	Juan Carlos Villar Herrera
Posición evaluada:	Desarrollador TI / Sénior
Repositorio (Módulo 2):	https://github.com/Juankvillar/seguros-bolivar
Stack tecnológico:	Java 21 | Spring Boot 3.4.3 | Maven | JUnit 5 | MockMvc
Fecha:	Noviembre 2026

Contenido del informe

- **Módulo 1 - Diseño de Sistema:** arquitectura de alto nivel, tres patrones justificados, modelo de datos, escalabilidad, observabilidad, tolerancia a fallos, versionamiento y diagrama de componentes con la integración CORE vía WebLogic.
- **Módulo 2 - Prueba Técnica Práctica:** API Spring Boot por capas con los seis endpoints, mock de CORE, API key y reglas de negocio; trazabilidad requisito - código - prueba.
- **Módulo 3 - Conocimientos en BBDD:** estrategias de optimización del JOIN entre orders (10 M) y customers (500 mil) filtrando México, con trade-offs y método de validación.
- **Módulo 4 - Conocimientos en Git:** incorporación de un cambio crítico de main a feature/new-login con git cherry-pick -x: comandos, justificación y variantes.
- **Módulo 5 - Liderazgo Técnico y Gestión:** respuesta a las cinco preguntas: prioridades, organización, métricas, prácticas obligatorias y manejo de la presión del negocio.

MATRIZ DE CUMPLIMIENTO Y SUPUESTOS GENERALES

RESUMEN

Trazabilidad entre el enunciado y este documento

Módulo	Lo que pide el enunciado	Dónde se responde
1 Diseño	Arquitectura de alto nivel de la plataforma (crear/consultar/modificar, renovación automática, notificaciones, CORE, 24/7).	Secciones 1.1, 1.2 y 1.3
1 Diseño	Seleccionar 3 patrones de arquitectura y justificarlos.	Sección 1.4
1 Diseño	Modelo de datos principal, sin SQL detallado.	Sección 1.5
1 Diseño	Escalabilidad, logs y observabilidad, tolerancia a fallos y versionamiento de APIs.	Sección 1.6
1 Diseño	Diagrama: Pólizas, Riesgos, Notificaciones, Adapter CORE, base de datos y API Gateway.	Sección 1.7 (incluye WebLogic y CORE)
2 Práctica	Spring Boot por capas, entidades Poliza y Riesgo, validaciones y README.	Secciones 2.1 y 2.2; README del repositorio
2 Práctica	Seis endpoints, mock POST /core-mock/evento y header x-api-key: 123456.	Sección 2.3
2 Práctica	Cuatro reglas de negocio esenciales.	Sección 2.4 (regla - código - prueba)
3 BBDD	Al menos tres estrategias para optimizar el JOIN orders / customers filtrando México.	Sección 3.2 (cinco estrategias)
4 Git	Comando o estrategia para llevar un cambio de main a feature/new-login y por qué.	Secciones 4.2 a 4.5
5 Liderazgo	Cinco preguntas: prioridades, organización, métricas, prácticas obligatorias y presión del negocio.	Preguntas 1 a 5

Supuestos generales

- **Motor de base de datos:** el enunciado no lo define. Se usa SQL estándar y se indican equivalentes Oracle donde aplica, dado que menciona PL/SQL y WebLogic.
- **IPC:** no se define fuente ni formato. En el ejercicio es un parámetro configurable (5 %) que el consumidor no puede modificar; en el diseño se propone una tabla de parámetros auditada.
- **Contrato del CORE:** se desconoce el contrato del servicio agnóstico de edición; se diseña un adapter con contrato canónico propio y se documenta la decisión síncrona o asíncrona.
- **Tecnologías de referencia:** broker de mensajes, motor de base de datos y librerías de resiliencia son opciones de referencia, no obligaciones del diseño.

MÓDULO 1 - DISEÑO DE SISTEMA: PLATAFORMA DE GESTIÓN DE PÓLIZAS

ARQUITECTURA

1.1 Contexto y reglas de dominio

Concepto	Regla del negocio
Póliza individual	Tomador y asegurado: el arrendatario. Beneficiario: el arrendador. Tiene exactamente un riesgo.
Póliza colectiva	Orientada a inmobiliarias y administraciones de copropiedad. Se asegura a los arrendatarios y los beneficiarios son los arrendadores. Tiene uno o muchos riesgos.
Valores	Toda póliza tiene periodo de vigencia, canon mensual de arrendamiento y prima. Prima = canon mensual x número de meses de la vigencia.
Renovación	Por el mismo periodo de vigencia inicial. El canon se ajusta según el incremento del IPC y la prima se recalcula con el nuevo canon.
Cancelación	Cancelar una póliza cancela todos sus riesgos. Una póliza cancelada no se renueva.
Integración	Toda acción que modifique el estado de pólizas o riesgos consume el servicio agnóstico de edición, expuesto por la capa media WebLogic, que mantiene actualizado el CORE.

1.2 Arquitectura de alto nivel

Se propone una arquitectura por componentes de negocio, con una puerta de entrada única y la integración con el CORE aislada en un adapter. Los componentes son lógicos: pueden desplegarse como servicios independientes o como un monolito modular inicial y separarse según carga y equipos, sin imponer microservicios desde el primer día.

Componente	Responsabilidad
API Gateway	Punto único de acceso para el front y otros consumidores: autenticación, límites de consumo (rate limiting), ruteo versionado y correlación de trazas.
Servicio de Pólizas	Crear, consultar y modificar pólizas; renovar (+IPC) y cancelar. Aplica las reglas de vigencia, canon y prima. Es la fuente de verdad del estado de la póliza.
Servicio de Riesgos	Consultar, agregar y cancelar riesgos. Valida las invariantes según el tipo de póliza (individual: 1; colectiva: 1..N).
Renovación automática	Proceso programado e idempotente: identifica pólizas próximas a vencer y solicita la renovación al Servicio de Pólizas, sin duplicar reglas de negocio.
Servicio de Notificaciones	Consume los eventos de creación y renovación confirmadas y envía correo o SMS con reintentos, sin bloquear la operación principal.
Adapter de integración con CORE	Capa anticorrupción (ACL): traduce el modelo del dominio al contrato del servicio de edición y gestiona idempotencia, timeouts y reintentos.
Capa media WebLogic y servicio agnóstico de edición	Ruta obligatoria de toda modificación de estado; actualiza el CORE transaccional legado.
Base de datos	Estado transaccional de pólizas, riesgos, participantes, renovaciones, bitácora de operaciones CORE y outbox de eventos.

1.3 Flujo de una modificación de estado (CORE vía WebLogic)

- **1.** El API Gateway autentica, aplica límites y enruta la solicitud al servicio dueño del recurso.
- **2.** El servicio valida las reglas de negocio (por ejemplo, una póliza cancelada no se renueva).
- **3.** Registra la operación con una clave de idempotencia y la invoca mediante el puerto de edición del CORE.
- **4.** El adapter traduce el mensaje y llama a la capa media WebLogic, que consume el servicio agnóstico de edición y actualiza el CORE.
- **5.** Con confirmación exitosa, el cambio de estado se confirma en la base de datos y, en la misma transacción, se escribe el evento en la tabla outbox.
- **6.** Un publicador entrega el evento al broker; Notificaciones lo consume y envía el correo o SMS.

Al cancelar una póliza también cambian sus riesgos: el cambio debe llegar al CORE mediante operaciones individuales o una operación compuesta, según el contrato del servicio agnóstico. Si el CORE no responde, la operación queda registrada como PENDIENTE con su clave de idempotencia para reintento controlado, o se devuelve un error recuperable (503), según el acuerdo de negocio con CORE. Nunca se pierde ni se duplica.

1.4 Tres patrones de arquitectura seleccionados

Patrón	Dónde se aplica	Por qué se justifica	Compromiso y mitigación
API Gateway	Entrada única para el front y los consumidores de pólizas y riesgos.	Centraliza seguridad, límites de consumo, trazabilidad y convivencia de versiones. El cliente no conoce la topología interna.	Es crítico y puede ser punto único de falla: se despliega redundado (activo-activo) y monitoreado.
Hexagonal (puertos y adaptadores)	Pólizas y Riesgos definen puertos de persistencia, CORE y eventos. WebLogic/CORE es un adaptador de salida.	Aísla las reglas de negocio de la infraestructura y del acoplamiento con el CORE legado; permite probar con dobles y cambiar la integración sin reescribir casos de uso.	Más interfaces y mapeos (DTOs). Se compensa con mantenibilidad. El Módulo 2 aplica esta separación con CoreGateway.
Orientada a eventos (con Outbox)	Las pólizas creadas o renovadas publican eventos que consume Notificaciones.	Desacopla la operación principal del envío de correo/SMS: se reintenta sin bloquear y escala por separado.	Consistencia eventual: exige consumidores idempotentes, cola de mensajes no procesables (DLQ) y monitoreo de eventos pendientes.

Considerados y descartados como patrón principal: CQRS (la carga de lectura actual no justifica modelos separados; se reevalúa si crecen los reportes) y microservicios completos (se separan por carga y equipo, no por defecto).

1.5 Modelo de datos principal

Entidad	Atributos clave	Relación y propósito
Poliza	id, tipo (INDIVIDUAL/COLECTIVA), estado (ACTIVA/RENOVADA/CANCELADA), fechaInicio, fechaFin, vigenciaMesesInicial, canonMensual, prima, version.	Prima = canon x vigencia. Conserva la vigencia inicial para renovar por el mismo periodo; version permite control optimista de concurrencia.
Riesgo	id, polizaId, estado (ACTIVO/CANCELADO), descripción y dirección del inmueble.	N:1 con Poliza. Individual: exactamente 1. Colectiva: 1..N.
Persona	id, tipoDocumento, numeroDocumento, nombre, correo, teléfono.	Reutilizable para arrendatarios, arrendadores, inmobiliarias y administraciones.
TomadorPoliza	polizaId, personaId.	Individual: el arrendatario. Colectiva: la inmobiliaria o administración de copropiedad.
ParticipanteRiesgo	riesgoId, personaId, rol (ASEGURADO/BENEFICIARIO).	Asegurado: arrendatario. Beneficiario: arrendador.
Renovacion	id, polizaId, vigencia, ipcAplicado, canonAnterior, canonNuevo, primaNueva, fecha, resultado.	Historial auditable de cada renovación.
ParametroIPC	periodo, tasa, fuente, vigenteDesde.	Evita tasas embebidas en el código y deja trazabilidad del ajuste.
OperacionCore	id, entidad, tipo, estadoSync (PENDIENTE/OK/ERROR), claveIdempotencia, intentos, respuesta, fechas.	Bitácora de lo enviado a WebLogic/CORE; sostiene los reintentos seguros.
EventoOutbox	id, tipo, payload, estado, intentos, fecha.	Publicación fiable de eventos de creación y renovación hacia Notificaciones.

```
Poliza 1 --- 1..N Riesgo
Poliza 1 --- 1..N TomadorPoliza N --- 1 Persona
Riesgo 1 --- 1..N ParticipanteRiesgo N --- 1 Persona
Poliza 1 --- 0..N Renovacion
Poliza / Riesgo 1 --- 0..N OperacionCore
Poliza 1 --- 0..N EventoOutbox
```

Índices por criterios frecuentes: (tipo, estado) para el listado, polizaId en Riesgo y estadoSync en OperacionCore. Las invariantes (individual = 1 riesgo; cancelada no se renueva) se validan en el dominio y no solo con restricciones de base de datos.

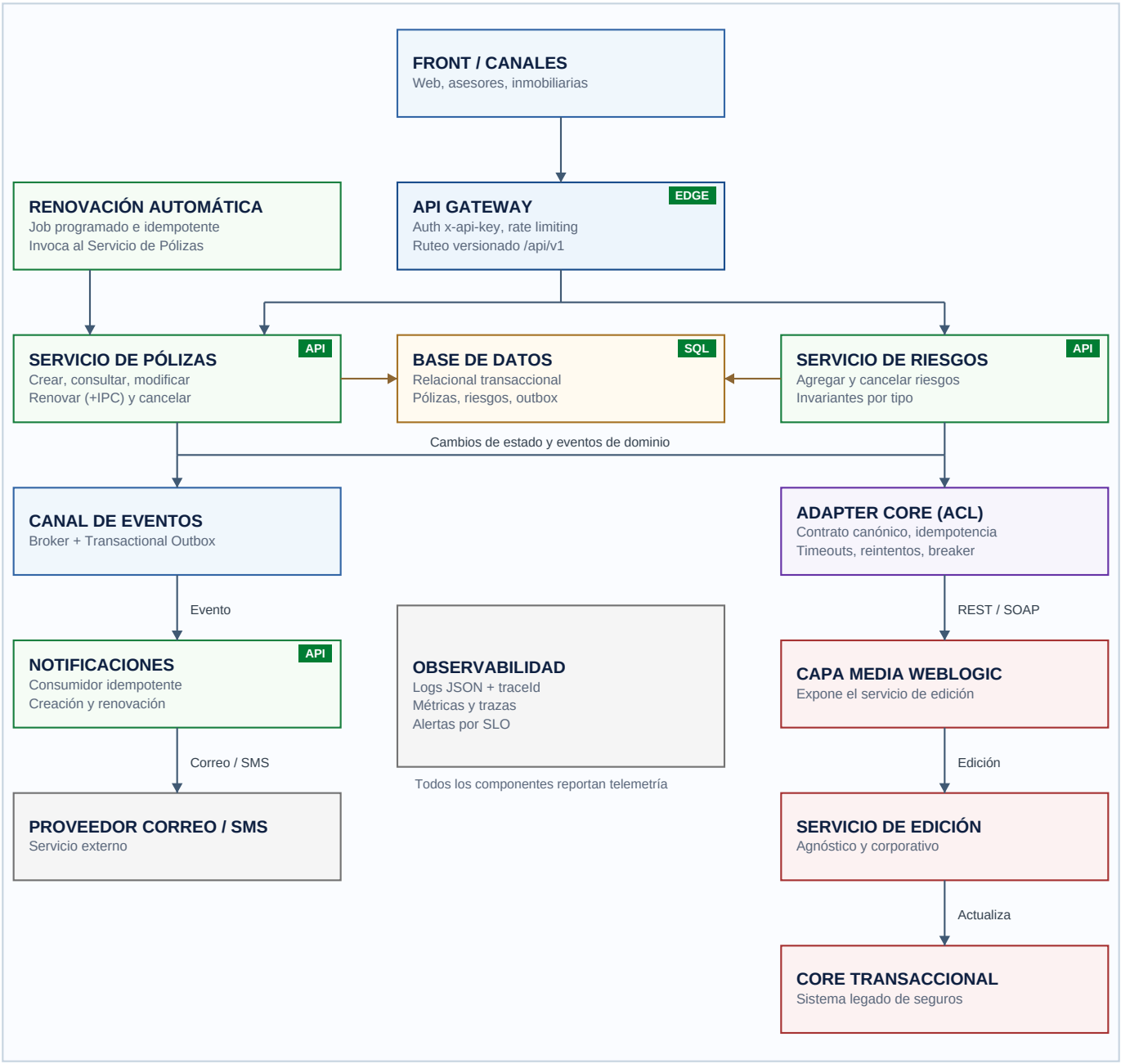
1.6 Escalabilidad, observabilidad, tolerancia a fallos y versionamiento

Tema	Estrategia propuesta
Escalabilidad	Servicios sin estado detrás del Gateway, con escalado horizontal automático (CPU y tasa de peticiones). Réplicas de lectura y caché para consultas frecuentes; consultas paginadas e indexadas. Notificaciones escala por consumidores del broker. El proceso de renovación evita ejecuciones duplicadas (bloqueo distribuido o ejecución única) y procesa por lotes. Límites de concurrencia hacia el CORE (bulkhead) para que la degradación del legado no afecte a toda la plataforma.
Logs y observabilidad	Logs estructurados (JSON) con identificador de correlación (traceId) desde el Gateway y sin datos personales innecesarios. Se registran polizaId, riesgoId, id de operación, resultado y latencia de las llamadas a WebLogic/CORE. Métricas de las cuatro señales (latencia, tráfico, errores, saturación), renovaciones procesadas, operaciones CORE pendientes y fallos de notificación. Trazas distribuidas hasta el adapter y alertas por SLO (disponibilidad, latencia p95, backlog del outbox).
Tolerancia a fallos y 24/7	Varias instancias en distintas zonas con health checks y despliegues sin caída (rolling o blue-green). Timeouts acotados, reintentos con backoff exponencial y jitter, y circuit breaker hacia WebLogic/CORE. Operaciones CORE idempotentes y reintentables mediante OperacionCore: una falla no duplica renovaciones, cancelaciones ni notificaciones. Outbox y DLQ para eventos. Respaldo y recuperación de la base de datos con RPO/RTO definidos y degradación controlada si el CORE no está disponible.
Versionamiento de APIs	Versión mayor en la ruta (/api/v1/polizas, /api/v1/riesgos). Los cambios compatibles son aditivos; los incompatibles crean /v2, que convive con v1 bajo una política de deprecación publicada. El Gateway enruta cada versión a su contrato. El contrato interno hacia WebLogic/CORE se versiona de forma independiente para aislar al front de cambios del legado.

MÓDULO 1 - 1.7 DIAGRAMA DE COMPONENTES

DIAGRAMA

1.7 Componentes principales e integración con el CORE



- **Camino síncrono:** Front, API Gateway, Servicio de Pólizas o Riesgos y base de datos. La renovación automática entra directamente al Servicio de Pólizas.
- **Camino asíncrono:** los cambios confirmados generan eventos (outbox, broker) que consume Notificaciones para enviar correo o SMS.
- **Integración con el CORE:** toda modificación de estado pasa por el Adapter (ACL), la capa media WebLogic y el servicio agnóstico de edición, que actualiza el CORE.

MÓDULO 2 - PRUEBA TÉCNICA PRÁCTICA (API DE GESTIÓN DE PÓLIZAS) GITHUB

2.1 Repositorio

Conforme al enunciado, el código se entrega por enlace al repositorio de GitHub, sin adjuntar carpetas locales ni archivos de Drive.

```
Repositorio: https://github.com/Juankvillar/seguros-bolivar
Rama main | Maven | Java 21 | Spring Boot 3.4.3 | README con ejecución, ejemplos curl y supuesto de IPC
```

2.2 Estructura del proyecto y capas

```
com.segurosbolivar.policies
 |- web         PolicyController, RiskController, CoreMockController, ApiKeyFilter
 |- service     PolicyService, CoreGateway, CoreMockService
 |- repository  PolicyRepository (interfaz), InMemoryPolicyRepository
 |- domain      Policy, Risk, PolicyType, PolicyStatus, RiskStatus
 |- dto         PolicyResponse, RiskResponse, AddRiskRequest, CoreEventRequest, CoreEventResponse, ErrorResponse
 |- exception   BusinessRuleViolationException, ResourceNotFoundException, ApiExceptionHandler
```

- **Entidades:** Policy y Risk (Poliza y Riesgo) encapsulan sus invariantes; la prima se calcula con BigDecimal (escala 2, HALF_UP).
- **Capas:** controladores delgados, reglas en el servicio y el dominio, persistencia tras una interfaz (PolicyRepository).
- **Validaciones y errores:** ApiExceptionHandler (@RestControllerAdvice) devuelve JSON homogéneo (timestamp, status, error, message, path): 404 recurso inexistente, 409 regla de negocio violada, 400 solicitud inválida.
- **Seguridad mínima:** ApiKeyFilter exige x-api-key: 123456 en /polizas, /riesgos y /core-mock; sin ella responde 401.

2.3 Endpoints implementados

Método	Endpoint	Comportamiento	Respuestas
GET	/polizas?tipo=&estado=	Lista pólizas; filtra por tipo (INDIVIDUAL/COLECTIVA) y estado (ACTIVA/RENOVADA/CANCELADA), ambos opcionales.	200
GET	/polizas/{id}/riesgos	Consulta los riesgos de la póliza.	200 / 404
POST	/polizas/{id}/renovar	Incrementa canon y prima en +IPC y pasa el estado a RENOVADA. No renueva pólizas canceladas.	200 / 404 / 409
POST	/polizas/{id}/cancelar	Cancela la póliza y todos sus riesgos.	200 / 404
POST	/polizas/{id}/riesgos	Agrega un riesgo; solo si la póliza es COLECTIVA y no está cancelada.	201 / 404 / 409
POST	/riesgos/{id}/cancelar	Cancela un riesgo puntual.	200 / 404
POST	/core-mock/evento	Mock de CORE: registra en logs el intento de envío (evento ACTUALIZACION, polizaId).	200 / 400

Las operaciones que modifican estado (renovar, cancelar póliza, agregar y cancelar riesgo) invocan coreGateway.notifyPolicyUpdated, que en el ejercicio queda registrado en el log del mock de CORE.

2.4 Reglas de negocio: regla, implementación y prueba

Regla del enunciado	Implementación	Prueba que la verifica
Individual con un solo riesgo	Policy (validación inicial) y PolicyService.addRisk (409 si no es colectiva).	enforcesTheSingleRiskInvariantForIndividualPolicies; rejectsAddingRiskToIndividualPolicy
No renovar una póliza cancelada	PolicyService.renew lanza BusinessRuleViolationException (409).	rejectsRenewalOfCancelledPolicy (servicio e integración)
Cancelar póliza cancela sus riesgos	Policy.cancel cancela cada riesgo.	cancellingPolicyCancelsAllItsRisks; cancelsPolicyAndCancelsAllItsAssociatedRisks
Agregar riesgo valida el tipo	PolicyService.addRisk valida tipo y estado.	addsRiskToCollectivePolicy; rejectsAddingRiskToIndividualPolicy
Renovar: +IPC, prima recalculada, estado RENOVADA	Policy.renew con la tasa app.policy.ipc-rate (5 %).	renewsPolicyUsingIpcAndRecalculatesPremium; renewsPolicySuccessfullyIncreasingRentAndPremium (canon 1.800.000 a 1.890.000; prima 22.680.000)
Header x-api-key obligatorio	ApiKeyFilter.	rejectsRequestsWithoutApiKey
Mock de CORE solo registra en logs	CoreMockService.	registersCoreMockEventSuccessfully; acceptsOnlyTheExpectedCoreUpdateEvent

2.5 Verificación y ejecución

Resultado obtenido al ejecutar mvn test sobre el repositorio (reportes surefire):

```
[INFO] Running com.segurosbolivar.policies.web.PolicyApiIntegrationTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.466 s
[INFO] Running com.segurosbolivar.policies.service.PolicyServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.061 s
[INFO] Results: Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
```

```
mvn test                    # 18 pruebas: 12 de integración (MockMvc) y 6 de servicio
mvn spring-boot:run         # API en http://localhost:8080
curl -H 'x-api-key: 123456' 'http://localhost:8080/polizas?tipo=COLECTIVA&estado=ACTIVA'
```

2.6 Alcance del ejercicio y evolución a producción

Tema	En el ejercicio	En producción
Persistencia	En memoria (ConcurrentHashMap), detrás de la interfaz PolicyRepository.	JPA con PostgreSQL u Oracle, control optimista por version y migraciones versionadas.
Concurrencia	Métodos de servicio sincronizados.	Transacciones y bloqueo optimista; sin bloqueo global.
IPC	Tasa fija configurable (5 %) en application.yml.	Tabla ParametroIPC alimentada desde la fuente oficial y auditada.
Integración CORE	CoreGateway y mock que solo registra en logs, como pide el enunciado.	Adapter hacia WebLogic con idempotencia, timeouts, reintentos, circuit breaker y outbox.
Seguridad	API key estática exigida por el enunciado.	OAuth2/JWT en el Gateway, secretos fuera del código y rate limiting.
Observabilidad	Logs de aplicación.	Logs JSON con traceId, métricas y trazas distribuidas.

El módulo práctico implementa lo esencial que pide el enunciado; la creación y modificación de pólizas por API se contempla en el diseño del Módulo 1.

MÓDULO 3 - CONOCIMIENTOS EN BBDD: OPTIMIZACIÓN DE SQL

SQL

3.1 Contexto y supuestos

Consulta frecuente y lenta: JOIN entre orders (10 millones de registros) y customers (500 mil) filtrando clientes de México. Se asume un motor relacional (se citan equivalentes Oracle), customers.customer_id como clave primaria y orders.customer_id como clave foránea. Antes de crear un índice se verifica con el plan de ejecución cuáles existen. El objetivo es reducir filas y lecturas antes de acceder a orders, la tabla de mayor volumen.

```
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'México';
```

3.2 Estrategias de optimización

- **Estrategia 1 - Índices alineados con el filtro y el JOIN.** Un índice compuesto en customers(country, customer_id) localiza los clientes de México sin recorrer 500 mil filas, y un índice en orders(customer_id) permite acceder a sus pedidos sin recorrer 10 millones (NESTED LOOPS). Es favorable cuando México es una fracción pequeña de los clientes; si es una proporción alta, un HASH JOIN con lectura secuencial puede costar menos, por eso se mide. Costo: almacenamiento y escritura más lenta.

```
CREATE INDEX ix_customers_country_custid ON customers (country, customer_id);
CREATE INDEX ix_orders_customer_id ON orders (customer_id);
```

- **Estrategia 2 - Índice de cobertura en orders.** Si la consulta es muy frecuente, un índice que incluya las columnas leídas permite resolverla solo desde el índice, sin acceder a la tabla. Es un índice más ancho: aumenta el costo de INSERT/UPDATE, por lo que se justifica con la frecuencia de uso. En PostgreSQL se expresa con INCLUDE.

```
CREATE INDEX ix_orders_cust_cover ON orders (customer_id, order_id, order_date, total_amount);
```

- **Estrategia 3 - Estadísticas actualizadas y lectura del plan.** Si la distribución de country es desigual, un histograma permite al optimizador estimar cuántos clientes devuelve el filtro y elegir bien el tipo de JOIN. Se compara filas estimadas contra reales en el plan; no se fuerza el plan con hints antes de comprobar un problema de cardinalidad.

```
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'CUSTOMERS', cascade => TRUE,
     method_opt => 'FOR COLUMNS SIZE 254 COUNTRY');
-- ejecutar la consulta con /*+ GATHER_PLAN_STATISTICS */ y luego:
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST'));
```

- **Estrategia 4 - Modelo de lectura materializado.** Si la consulta se repite y se tolera un refresco controlado, una vista materializada precombina pedidos y clientes y evita repetir el JOIN. El costo es almacenamiento, trabajo de refresco y posible desactualización. REFRESH FAST ON COMMIT exige logs de vista materializada y ROWID en el SELECT; con refresco programado se usa COMPLETE ON DEMAND. Antes se confirma si el país debe ser el actual del cliente o el del momento del pedido.

```
CREATE MATERIALIZED VIEW mv_orders_mexico BUILD IMMEDIATE REFRESH COMPLETE ON DEMAND AS
SELECT o.order_id, o.order_date, o.customer_id, c.customer_name, o.total_amount
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'México';
```

- **Estrategia 5 - Limitar lo que se transporta.** En consumo interactivo no se devuelven todos los pedidos de México: se seleccionan solo las columnas necesarias y se pagina por clave (keyset) con orden determinista, en lugar de OFFSET alto. No sustituye los índices ni aplica a exportaciones completas, pero reduce red, memoria y serialización.

```
... WHERE c.country = 'México' AND o.order_id > :ultimo_id
ORDER BY o.order_id FETCH FIRST 50 ROWS ONLY;
```

3.3 Lo que no haría sin evidencia

- **Particionar orders por fecha como solución principal:** la consulta no filtra por fecha, así que no habría eliminación de particiones; es útil para retención y otros patrones de consulta.
- **Forzar el plan con hints:** pueden degradar la consulta cuando cambien el volumen o la distribución de los datos.
- **Texto libre en el filtro:** comparar 'México' como cadena es frágil (acentos, mayúsculas); un código estable (country_code = 'MX') o una tabla countries con clave foránea reduce errores y el tamaño del índice.
- **PL/SQL:** encapsular la consulta o usar variables bind reduce hard parses, pero no acelera el JOIN: la mejora viene del plan, las estadísticas, los índices y el modelo físico.

3.4 Validación

Se toma una línea base en un ambiente con volumen y distribución similares a producción y se repite con los mismos parámetros después de cada cambio, uno a la vez. Se miden: tiempo de respuesta, CPU, lecturas lógicas y físicas, uso de espacio temporal, tipo de JOIN y filas estimadas frente a reales. También se mide el impacto de cada índice o refresco sobre las escrituras, y se descarta lo que no aporte.

MÓDULO 4 - CONOCIMIENTOS EN GIT: INCORPORAR UN CAMBIO PUNTUAL

GIT

4.1 Escenario

Trabajo en feature/new-login. Un compañero fusionó en main un cambio crítico que corrige un bug de seguridad. Necesito incorporar ese cambio específico en mi rama, urgentemente, sin traer el resto de las actualizaciones de main.

4.2 Estrategia elegida: git cherry-pick -x

git cherry-pick aplica el diff de un commit puntual sobre la rama actual y crea un commit nuevo. No incorpora la historia ni los cambios no relacionados de main, que es exactamente lo que se pide. La opción -x agrega al mensaje la línea (cherry picked from commit ...), lo que deja trazabilidad: en una revisión o auditoría se identifica de qué corrección de main proviene el cambio.

4.3 Procedimiento y justificación

Paso	Comando	Justificación
1	git status (git stash push -u si hay trabajo sin confirmar)	Árbol limpio: evita mezclar el trabajo en curso con el cherry-pick.
2	git fetch origin	Actualiza las referencias remotas para ver el commit ya fusionado en main.
3	git log --oneline --first-parent origin/main -n 15 ; git show --stat <sha>	Ubica el commit exacto de la corrección y verifica su contenido, no solo el título.
4	git switch feature/new-login	Se posiciona en la rama que necesita el cambio.
5	git cherry-pick -x <sha>	Aplica solo ese cambio, con referencia al commit original.
6	En conflicto: git status ; resolver ; git add <archivos> ; git cherry-pick --continue	Se resuelve conservando el comportamiento seguro y compatible con el login. Si no se puede adaptar con seguridad: git cherry-pick --abort.
7	git show --stat HEAD ; mvn test	Valida el resultado y ejecuta las pruebas, más las verificaciones de seguridad aplicables, antes de publicar.
8	git push origin feature/new-login	Publica el cambio; con ramas protegidas se abre una solicitud de cambio para revisión.

4.4 Variantes según cómo se fusionó el cambio en main

Caso	Qué se hace
Squash merge o commit único	git cherry-pick -x <sha> del commit resultante en main.
Varios commits (rebase o fast-forward)	git cherry-pick -x <sha-primero>^..<sha-ultimo> aplica el rango en orden, o se listan los sha uno por uno.
Merge commit (--no-ff)	Se prefieren los commits individuales de la rama fusionada. Si solo existe el merge commit: git cherry-pick -x -m 1 <sha-merge>, donde -m 1 toma main como padre de referencia; se revisa antes con git show.
Después, al integrar feature/new-login en main	El cambio ya existe con el mismo diff; Git suele reconciliarlo sin duplicarlo ni generar conflicto. Se revisa en el pull request.

4.5 Alternativas descartadas

Alternativa	Motivo
git merge origin/main	Incorpora todos los commits acumulados en main; contradice la necesidad de traer solo la corrección.
git rebase origin/main	Reescribe la rama sobre todo main y también incorpora todo; además reescribe la historia, riesgoso si la rama es compartida.
git checkout origin/main -- <archivo>	Trae el archivo completo de main (con otros cambios), no el diff del commit; sin referencia al commit original.
Copiar archivos manualmente	Pierde trazabilidad, puede omitir parte de la corrección y dificulta auditar qué cambio se validó.

MÓDULO 5 - LIDERAZGO TÉCNICO Y GESTIÓN

LIDERAZGO

Contexto y enfoque

Equipo de 8 desarrolladores con 40 % de deuda técnica en servicios clave, 10 incidentes críticos en el último mes, presión para entregar una funcionalidad en 3 semanas, falta de estándares con code review irregular y dos desarrolladores junior con brechas técnicas importantes. El objetivo no es eliminar el 40 % de deuda en dos semanas: es estabilizar el servicio, reducir el riesgo de una nueva interrupción y acordar un alcance de entrega verificable y sostenible.

Pregunta 1 - ¿Cuáles serían tus 5 prioridades en las primeras 2 semanas?

Prioridad	Acciones	Resultado esperado
1. Estabilizar los servicios críticos	Clasificar los 10 incidentes por impacto, causa raíz y recurrencia; definir guardia, escalamiento, runbooks y reversión; análisis post-incidente sin culpas; corregir primero lo que afecte disponibilidad, seguridad o integridad de datos.	Menor recurrencia y tiempos de restauración conocidos.
2. Acordar un MVP seguro con negocio	Dividir la funcionalidad en cortes verticales priorizados y explicitar qué cabe en 3 semanas con controles de calidad. Lo no esencial se programa después, no se absorbe como trabajo no planificado.	Alcance verificable y fecha creíble.
3. Atacar la deuda de alto riesgo	Reservar capacidad (30 a 35 %) para la deuda ligada a los incidentes: dependencias obsoletas, falta de pruebas en rutas críticas y observabilidad insuficiente. Se prioriza por riesgo e impacto, no por antigüedad.	Causas raíz mitigadas.
4. Instalar controles mínimos	Desde la primera semana: línea base de pruebas, code review obligatorio, integración continua, despliegue reversible, monitoreo y definición de terminado.	Calidad incorporada al flujo, no al final.
5. Aclarar propiedad y fortalecer al equipo	Responsable técnico por servicio, decisiones clave documentadas (registros breves de decisión) y pair programming de los dos junior con quienes conocen los componentes críticos.	Menos conocimiento concentrado.

Secuencia: semana 1 para triage, controles mínimos y acuerdo del MVP; semana 2 para las primeras correcciones de causa raíz, la primera demostración al negocio y la línea base de métricas.

Pregunta 2 - ¿Cómo organizarías al equipo para mejorar velocidad y calidad?

- **Dos células temporales coordinadas, no aisladas:** Entrega (4 personas, enfocada en el MVP en cortes verticales con pruebas, observabilidad básica y ruta de reversión) y Confiabilidad (4 personas, enfocada en estabilizar los servicios de mayor impacto, eliminar causas recurrentes y preparar los controles de despliegue).
- **Coordinación:** misma definición de terminado, tablero de riesgos compartido y sincronización diaria breve. Si aparece un incidente crítico, la capacidad se reasigna de forma explícita con negocio, en lugar de absorberlo con horas extra sostenidas.
- **Code review:** por rotación, con al menos una persona conocedora del dominio en cambios sensibles.
- **Perfiles junior:** tareas acotadas, pair programming y revisiones guiadas; se entrega valor mientras se reduce el riesgo de conocimiento concentrado. Los integrantes rotan entre células cada sprint.

Pregunta 3 - ¿Qué métricas implementarías para evaluar el desempeño del área?

Dimensión	Métricas	Para qué sirven
Confiabilidad	Incidentes P1/P2, tiempo medio de restauración (MTTR), recurrencia y cumplimiento de SLO y alertas.	Confirmar que la estabilización reduce el impacto real, no solo tickets cerrados.
Flujo de entrega	Lead time, cycle time, throughput, trabajo en curso (WIP) e historias terminadas frente al alcance acordado.	Detectar cuellos de botella y prever con honestidad el cumplimiento del MVP.
Calidad y seguridad	Cobertura de rutas críticas, defectos escapados, fallos de CI y hallazgos de seguridad abiertos.	Evitar que la velocidad se obtenga trasladando riesgo a producción.
Salud operativa	Frecuencia de despliegue, tasa de cambios fallidos y de reversión, latencia y errores de servicios críticos (métricas DORA).	Verificar que el sistema se mantiene sano durante la entrega.
Deuda técnica	Elementos de alto riesgo resueltos o mitigados, antigüedad y tendencia del backlog.	Mantener visible el riesgo que no se resuelve en este ciclo.
Equipo	Tiempo de espera de revisiones de PR, carga de guardia y avance de los perfiles junior.	Cuidar la sostenibilidad y el aprendizaje.

Primero se mide la línea base y luego se fijan metas con el equipo (por ejemplo, reducir MTTR y llevar los incidentes P1 a cero). Las métricas se leen por tendencia y contexto; no se usan para evaluar personas ni sustituyen el análisis de incidentes.

Pregunta 4 - ¿Qué prácticas técnicas establecerías como obligatorias?

- **Control de cambios:** ramas protegidas, solicitudes de cambio pequeñas con revisión obligatoria (una aprobación; dos en cambios críticos) y estándares de código con linters.
- **Integración continua:** compilación, pruebas automatizadas, análisis estático y escaneo de dependencias y secretos antes de integrar.
- **Pruebas:** automatización priorizada en las rutas críticas, con pruebas unitarias, de integración y de contrato.
- **Definición de terminado:** pruebas, documentación operativa mínima, monitoreo, registro de cambios y plan de reversión cuando aplique.

- **Despliegue seguro:** liberaciones graduales con interruptores de funcionalidad (feature flags) y reversión probada.
- **Observabilidad:** logs estructurados, métricas, trazabilidad, tableros y alertas accionables en los servicios críticos.
- **Gestión de incidentes:** niveles de severidad, responsable, comunicación, procedimiento de reversión y análisis posterior sin culpas con acciones verificables.
- **Decisiones:** registro breve de decisiones de arquitectura y priorización visible de la deuda técnica.

En una corrección de emergencia se puede acortar el flujo, pero no eliminar los controles esenciales: debe haber responsable, evidencia de validación, trazabilidad y una revisión posterior.

Pregunta 5 - ¿Cómo gestionarías la presión del negocio sin comprometer la calidad?

- **Datos y alternativas, no un "no" genérico:** se presentan opciones (un MVP que cabe en 3 semanas frente al alcance completo con su fecha y riesgo), el costo de los 10 incidentes y qué se pospone para proteger disponibilidad y seguridad.
- **Hitos semanales demostrables:** una demostración por semana y un tablero de riesgos compartido con producto para sostener la confianza.
- **Regla acordada de antemano:** un incidente crítico o una dependencia nueva relevante obliga a revisar alcance, capacidad o fecha.
- **Exposición controlada:** liberaciones graduales e interruptores de funcionalidad permiten validar valor con riesgo acotado.
- **Qué no se hace:** normalizar horas extra ni suprimir controles. Se hace visible el costo de cada decisión, se protege la capacidad de recuperación del equipo y negocio participa en las decisiones de alcance y riesgo.